

JAVA DAY 2 PLACEMENT PREPARATION GUIDE

Arrays • Array Operations • 2D Arrays • Strings • StringBuilder/Buffer
Methods • Recursion • Class & Object • Constructors • this / static / final
Encapsulation • Inheritance (Single, Multilevel, Hierarchical)

Complete Concept Explanations + Two Program Examples for Every Topic

Table of Contents

1. 1D Arrays – Traversal
2. Array Operations – Insert, Delete, Search
3. 2D Arrays – Matrix Traversal
4. Matrix Operations – Transpose
5. String Basics – Immutable Nature
6. String Methods – API Usage
7. StringBuilder – Mutable Strings
8. StringBuffer – Thread Safety
9. Advanced Strings – Pattern Matching
10. Methods & its types – Creation
11. Pass by Value – Parameter Passing
12. Recursion Intro – Call Stack
13. Class & Object – Class Structure, Object Creation
14. Constructors – Default, Parameterized
15. this Keyword – Instance Variables, Constructor Chaining
16. static Keyword – Variables, Methods, Blocks
17. final Keyword – Variable, Method, Class
18. Encapsulation – Getters & Setters
19. Inheritance – Single Inheritance
20. Inheritance types – Multilevel & Hierarchical

1. 1D Arrays – Traversal

An array is a fixed-size, contiguous collection of elements of the **same type**. In Java, arrays are objects. Index starts at 0. Length is fixed after creation (accessed via `.length`).

Declaration & Creation

- `int[] arr;` or `int arr[];` (declaration)
- `arr = new int[5];` (creation – default values 0)
- `int[] arr = {10, 20, 30};` (declaration + initialization)

Traversal

Visiting every element exactly once – usually with a classic for loop or enhanced for-each loop.

Example 1: Declaration, initialization & classic traversal

```
public class ArrayTraversal1 {
    public static void main(String[] args) {
        int[] marks = {85, 90, 78, 92, 88};
        System.out.println("Length = " + marks.length);
        System.out.print("Marks: ");
        for (int i = 0; i < marks.length; i++) {
            System.out.print(marks[i] + " ");
        }
        System.out.println();
    }
}
// Output: Length = 5
// Marks: 85 90 78 92 88
```

Example 2: Enhanced for-each + sum & average

```
public class ArrayTraversal2 {
    public static void main(String[] args) {
        double[] temps = {28.5, 30.1, 27.8, 29.0, 31.2};
        double sum = 0;
        for (double t : temps) {           // for-each
            sum += t;
        }
        System.out.println("Sum = " + sum);
        System.out.println("Average = " + (sum / temps.length));
    }
}
// Output: Sum = 146.6
// Average = 29.32
```

2. Array Operations – Insert, Delete, Search

Because arrays have fixed size, insert/delete usually require shifting elements or using a logical size. Search can be linear ($O(n)$) or binary ($O(\log n)$ on sorted arrays).

- **Linear Search** – check each element until found or end.
- **Insert** – shift elements right from the insertion index, then place new value.
- **Delete** – shift elements left from the deletion index (overwrite).

Example 1: Linear Search + Insert (with logical size)

```
public class ArrayOps1 {
    static int linearSearch(int[] a, int n, int key) {
        for (int i = 0; i < n; i++)
            if (a[i] == key) return i;
        return -1;
    }
    static int insert(int[] a, int n, int pos, int val) {
        if (n >= a.length || pos < 0 || pos > n) return n;
        for (int i = n; i > pos; i--) a[i] = a[i - 1];
        a[pos] = val;
        return n + 1;
    }
    public static void main(String[] args) {
        int[] a = new int[10];
        a[0]=10; a[1]=20; a[2]=30; a[3]=40;
        int n = 4;
        System.out.println("Index of 30: " + linearSearch(a, n, 30));
        n = insert(a, n, 2, 25);          // insert 25 at index 2
        for (int i = 0; i < n; i++) System.out.print(a[i] + " ");
    }
}
// Output: Index of 30: 2
// 10 20 25 30 40
```

Example 2: Delete element by value

```
public class ArrayOps2 {
    static int delete(int[] a, int n, int key) {
        int idx = -1;
        for (int i = 0; i < n; i++)
            if (a[i] == key) { idx = i; break; }
        if (idx == -1) return n;          // not found
        for (int i = idx; i < n - 1; i++) a[i] = a[i + 1];
        return n - 1;
    }
    public static void main(String[] args) {
        int[] a = {5, 10, 15, 20, 25};
        int n = 5;
        n = delete(a, n, 15);
        for (int i = 0; i < n; i++) System.out.print(a[i] + " ");
    }
}
// Output: 5 10 20 25
```

3. 2D Arrays – Matrix Traversal

A 2D array is an array of arrays (matrix). Declaration: `int[][] mat = new int[rows][cols];` or `int[][] mat = {{1,2},{3,4}};`. Access element with `mat[i][j]`.

Common traversals: row-major (outer row, inner column), column-major, diagonal, spiral, etc.

Example 1: Create & print matrix (row-major)

```
public class MatrixTraversal1 {
    public static void main(String[] args) {
        int[][] mat = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
        int rows = mat.length;
        int cols = mat[0].length;
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                System.out.print(mat[i][j] + " ");
            }
            System.out.println();
        }
    }
}
// Output:
// 1 2 3
// 4 5 6
// 7 8 9
```

Example 2: Sum of each row & column

```
public class MatrixTraversal2 {
    public static void main(String[] args) {
        int[][] m = {{10,20,30},{40,50,60},{70,80,90}};
        int r = m.length, c = m[0].length;
        for (int i = 0; i < r; i++) {
            int rowSum = 0;
            for (int j = 0; j < c; j++) rowSum += m[i][j];
            System.out.println("Row " + i + " sum = " + rowSum);
        }
        for (int j = 0; j < c; j++) {
            int colSum = 0;
            for (int i = 0; i < r; i++) colSum += m[i][j];
            System.out.println("Col " + j + " sum = " + colSum);
        }
    }
}
```

4. Matrix Operations – Transpose

Transpose of a matrix swaps rows with columns: element at (i,j) moves to (j,i). For an m×n matrix the transpose is n×m. For a square matrix it can be done in-place.

Example 1: Transpose of a rectangular matrix

```
public class Transpose1 {
    public static void main(String[] args) {
        int[][] a = {{1,2,3},{4,5,6}}; // 2x3
        int rows = a.length, cols = a[0].length;
        int[][] t = new int[cols][rows]; // 3x2
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                t[j][i] = a[i][j];
        for (int[] row : t) {
            for (int val : row) System.out.print(val + " ");
            System.out.println();
        }
    }
}
// Output:
// 1 4
// 2 5
// 3 6
```

Example 2: In-place transpose of square matrix

```
public class Transpose2 {
    public static void main(String[] args) {
        int[][] a = {{1,2,3},{4,5,6},{7,8,9}};
        int n = a.length;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                int temp = a[i][j];
                a[i][j] = a[j][i];
                a[j][i] = temp;
            }
        }
        for (int[] row : a) {
            for (int v : row) System.out.print(v + " ");
            System.out.println();
        }
    }
}
// Output:
// 1 4 7
// 2 5 8
// 3 6 9
```

5. String Basics – Immutable Nature

In Java, **String is immutable** – once created, its content cannot be changed. Any “modification” (concat, replace, etc.) actually creates a **new** String object. Strings are stored in the String Pool (part of Heap). Literals are interned; `new String("x")` creates a new object even if the content exists in the pool.

Why immutable? Security, thread-safety, caching (hashCode), and String Pool optimization.

Example 1: Demonstrating immutability

```
public class StringImmutable1 {
    public static void main(String[] args) {
        String s1 = "Hello";
        String s2 = s1;           // same reference
        s1 = s1 + " World";       // new object created
        System.out.println("s1 = " + s1); // Hello World
        System.out.println("s2 = " + s2); // Hello (unchanged)
        System.out.println(s1 == s2);    // false
    }
}
```

Example 2: String Pool vs new

```
public class StringImmutable2 {
    public static void main(String[] args) {
        String a = "Java";
        String b = "Java";           // same pool object
        String c = new String("Java"); // new heap object
        System.out.println(a == b);   // true (same reference)
        System.out.println(a == c);   // false (different objects)
        System.out.println(a.equals(c)); // true (same content)
        System.out.println(a == c.intern()); // true after interning
    }
}
```

6. String Methods – API Usage

Important String methods (all return a new String unless noted): `length()`, `charAt(i)`, `substring(begin)`, `substring(begin,end)`, `indexOf`, `lastIndexOf`, `contains`, `startsWith`, `endsWith`, `toLowerCase`, `toUpperCase`, `trim`, `replace`, `split`, `equals`, `equalsIgnoreCase`, `compareTo`, `isEmpty`, `isBlank` (Java 11+), `valueOf`, etc.

Example 1: Common methods

```
public class StringMethods1 {
    public static void main(String[] args) {
        String s = " Hello Java ";
        System.out.println("length = " + s.length());
        System.out.println("trim    = [" + s.trim() + "]");
        System.out.println("charAt(2) = " + s.charAt(2));
        System.out.println("substring(2,7) = " + s.substring(2, 7));
        System.out.println("indexOf('J') = " + s.indexOf('J'));
        System.out.println("contains Java? " + s.contains("Java"));
        System.out.println("toUpperCase = " + s.toUpperCase());
        System.out.println("replace = " + s.replace("Java", "World"));
    }
}
```

Example 2: split & compareTo

```
public class StringMethods2 {
    public static void main(String[] args) {
        String csv = "apple,banana,cherry";
        String[] fruits = csv.split(",");
        for (String f : fruits) System.out.println(f);

        String a = "apple", b = "banana";
        System.out.println(a.compareTo(b));    // negative (a < b)
        System.out.println(a.equalsIgnoreCase("APPLE")); // true
    }
}
```

7. StringBuilder – Mutable Strings

StringBuilder is a **mutable** sequence of characters. It is not synchronized (not thread-safe) and is faster than StringBuffer for single-threaded use. Preferred when you need many modifications (append, insert, delete, reverse) without creating many intermediate String objects.

Key methods: append(), insert(), delete(), deleteCharAt(), reverse(), toString(), setCharAt(), length(), capacity().

Example 1: Building a string efficiently

```
public class StringBuilder1 {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder();
        sb.append("Hello");
        sb.append(" ");
        sb.append("World");
        sb.insert(5, ",");
        System.out.println(sb);           // Hello, World
        sb.reverse();
        System.out.println(sb);           // dlrow ,olleH
        System.out.println(sb.toString());
    }
}
```

Example 2: Performance idea – concat in loop

```
public class StringBuilder2 {
    public static void main(String[] args) {
        // Bad: many intermediate Strings
        // String s = "";
        // for (int i = 0; i < 1000; i++) s += i;

        // Good: single mutable buffer
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 10; i++) {
            sb.append(i).append(" ");
        }
        System.out.println(sb.toString().trim());
    }
}
// Output: 0 1 2 3 4 5 6 7 8 9
```

8. StringBuffer – Thread Safety

StringBuffer is almost identical to StringBuilder but **synchronized** – all public methods are thread-safe. Use it when multiple threads may modify the same buffer. In single-threaded code prefer StringBuilder (faster).

Example 1: Basic StringBuffer usage

```
public class StringBuffer1 {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Java");
        sb.append(" Programming");
        sb.insert(4, " Language");
        System.out.println(sb);          // Java Language Programming
        sb.delete(5, 14);
        System.out.println(sb);          // Java Programming
        sb.reverse();
        System.out.println(sb);
    }
}
```

Example 2: Capacity & ensureCapacity

```
public class StringBuffer2 {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer();
        System.out.println("Default capacity: " + sb.capacity()); // 16
        sb.append("HelloWorld12345"); // 15 chars
        System.out.println("After append: " + sb.capacity());
        sb.ensureCapacity(50);
        System.out.println("After ensureCapacity(50): " + sb.capacity());
        // formula when exceeded: (oldCapacity * 2) + 2
    }
}
```

9. Advanced Strings – Pattern Matching

Pattern matching with strings can be done with: `indexOf` / `contains` (simple), regex via `String.matches()`, `split()`, or the full `java.util.regex` package (`Pattern` + `Matcher`). Common interview patterns: anagrams, palindrome, frequency count, longest substring, etc.

Example 1: Palindrome check & simple pattern

```
public class Pattern1 {
    static boolean isPalindrome(String s) {
        int i = 0, j = s.length() - 1;
        while (i < j) {
            if (s.charAt(i) != s.charAt(j)) return false;
            i++; j--;
        }
        return true;
    }
    public static void main(String[] args) {
        System.out.println(isPalindrome("radar"));    // true
        System.out.println(isPalindrome("java"));    // false
        String email = "user@example.com";
        System.out.println(email.matches(".*@.*\\..*")); // simple regex
    }
}
```

Example 2: Character frequency (pattern / counting)

```
public class Pattern2 {
    public static void main(String[] args) {
        String s = "programming";
        int[] freq = new int[256];
        for (char c : s.toCharArray()) freq[c]++;
        for (int i = 0; i < 256; i++) {
            if (freq[i] > 0)
                System.out.println((char)i + " : " + freq[i]);
        }
    }
}
```

10. Methods & its types – Creation

A method is a block of code that performs a specific task. Syntax:

```
accessModifier returnType methodName(parameters) { body }
```

Types: **Instance methods** (need object), **Static methods** (belong to class), **Concrete** vs **Abstract**, methods with/without return value, with/without parameters, varargs, overloaded methods (same name, different parameter list).

Example 1: Method creation – with & without return / params

```
public class Methods1 {  
    // no param, no return  
    static void greet() {  
        System.out.println("Hello!");  
    }  
    // with params, with return  
    static int add(int a, int b) {  
        return a + b;  
    }  
    // instance method  
    void display(String msg) {  
        System.out.println(msg);  
    }  
    public static void main(String[] args) {  
        greet();  
        System.out.println(add(5, 7));  
        new Methods1().display("Instance method called");  
    }  
}
```

Example 2: Method overloading

```
public class Methods2 {  
    static int max(int a, int b) { return (a > b) ? a : b; }  
    static double max(double a, double b) { return (a > b) ? a : b; }  
    static int max(int a, int b, int c) {  
        return max(max(a, b), c);  
    }  
    public static void main(String[] args) {  
        System.out.println(max(10, 20));  
        System.out.println(max(3.5, 2.1));  
        System.out.println(max(5, 9, 3));  
    }  
}
```

11. Pass by Value – Parameter Passing

Java is **strictly pass-by-value**. For primitives the value is copied. For objects the **reference value** (address) is copied – so the method receives a copy of the reference. Reassigning the parameter does not change the caller's reference, but modifying the object's state does.

Example 1: Primitive vs object state change

```
class Box { int value; Box(int v) { value = v; } }

public class PassByValue1 {
    static void changePrimitive(int x) { x = 100; }
    static void changeObject(Box b) { b.value = 100; }
    static void reassign(Box b) { b = new Box(200); }

    public static void main(String[] args) {
        int n = 10;
        changePrimitive(n);
        System.out.println(n);           // 10 (unchanged)

        Box box = new Box(10);
        changeObject(box);
        System.out.println(box.value);    // 100 (state changed)

        reassign(box);
        System.out.println(box.value);    // still 100 (reassign local only)
    }
}
```

Example 2: Swapping – why swap fails for objects' references

```
public class PassByValue2 {
    static void swap(int a, int b) {
        int t = a; a = b; b = t;
    }

    public static void main(String[] args) {
        int x = 5, y = 10;
        swap(x, y);
        System.out.println(x + " " + y); // 5 10 – no swap
        // To swap object contents you must change fields, not the references.
    }
}
```

12. Recursion Intro – Call Stack

Recursion is a method calling itself. Every call pushes a new frame on the **call stack** (local variables, parameters, return address). A base case is mandatory to stop recursion; otherwise `StackOverflowError` occurs.

Example 1: Factorial (classic recursion)

```
public class Recursion1 {
    static int factorial(int n) {
        if (n <= 1) return 1;           // base case
        return n * factorial(n - 1);    // recursive call
    }
    public static void main(String[] args) {
        System.out.println(factorial(5)); // 120
        // Call stack grows: f(5)→f(4)→f(3)→f(2)→f(1) then unwinds
    }
}
```

Example 2: Print numbers 1 to N using recursion

```
public class Recursion2 {
    static void print1ToN(int n) {
        if (n == 0) return;             // base
        print1ToN(n - 1);
        System.out.print(n + " ");
    }
    public static void main(String[] args) {
        print1ToN(5); // 1 2 3 4 5
    }
}
```

13. Class & Object – Class Structure, Object Creation

A **class** is a blueprint. An **object** is an instance of a class. Class contains fields (state) and methods (behavior). Object creation: `ClassName obj = new ClassName();` (or with constructor arguments). Multiple objects can be created from one class.

Example 1: Simple class & object

```
class Student {
    String name;
    int roll;
    void display() {
        System.out.println(name + " - " + roll);
    }
}

public class ClassObject1 {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "Alice";
        s1.roll = 101;
        s1.display();

        Student s2 = new Student();
        s2.name = "Bob";
        s2.roll = 102;
        s2.display();
    }
}
```

Example 2: Multiple objects & default values

```
class Counter {
    int count;           // default 0
    void increment() { count++; }
}

public class ClassObject2 {
    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();
        c1.increment();
        c1.increment();
        c2.increment();
        System.out.println(c1.count); // 2
        System.out.println(c2.count); // 1
    }
}
```

14. Constructors – Default, Parameterized

A constructor initializes a new object. It has the **same name as the class** and no return type. • **Default constructor** – no parameters (compiler provides one if you write none).

- **Parameterized constructor** – takes arguments to initialize fields.
- Constructor overloading is common. If you write any constructor, the compiler does **not** add a default one.

Example 1: Default + Parameterized

```
class Person {
    String name;
    int age;
    // Default constructor
    Person() {
        name = "Unknown";
        age = 0;
    }
    // Parameterized
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    void show() { System.out.println(name + " (" + age + ")"); }
}

public class Constructor1 {
    public static void main(String[] args) {
        Person p1 = new Person();
        Person p2 = new Person("Alice", 22);
        p1.show(); // Unknown (0)
        p2.show(); // Alice (22)
    }
}
```

Example 2: Constructor overloading

```
class Rectangle {
    int length, breadth;
    Rectangle() { length = breadth = 1; }
    Rectangle(int side) { length = breadth = side; }
    Rectangle(int l, int b) { length = l; breadth = b; }
    int area() { return length * breadth; }
}

public class Constructor2 {
    public static void main(String[] args) {
        System.out.println(new Rectangle().area()); // 1
        System.out.println(new Rectangle(5).area()); // 25
        System.out.println(new Rectangle(4, 6).area()); // 24
    }
}
```

15. this Keyword – Instance Variables, Constructor Chaining

this is a reference to the **current object**. Uses:

1. Distinguish instance variable from parameter (same name).
2. Call another constructor of the same class (constructor chaining) – must be first statement.
3. Pass current object as argument / return current object.

Example 1: Resolving name conflict

```
class Employee {
    String name;
    double salary;
    Employee(String name, double salary) {
        this.name = name;        // this.name = instance var
        this.salary = salary;
    }
    void display() {
        System.out.println(this.name + " : " + this.salary);
    }
}
public class This1 {
    public static void main(String[] args) {
        new Employee("Ravi", 50000).display();
    }
}
```

Example 2: Constructor chaining with this()

```
class Box {
    int l, b, h;
    Box() { this(1, 1, 1); }           // chain
    Box(int s) { this(s, s, s); }
    Box(int l, int b, int h) {
        this.l = l; this.b = b; this.h = h;
    }
    int volume() { return l * b * h; }
}
public class This2 {
    public static void main(String[] args) {
        System.out.println(new Box().volume());    // 1
        System.out.println(new Box(3).volume());   // 27
        System.out.println(new Box(2, 3, 4).volume()); // 24
    }
}
```

16. static Keyword – Variables, Methods, Blocks

- **static variable** – belongs to the class; shared by all instances; one copy.
- **static method** – can be called without an object; can access only static members directly.
- **static block** – runs once when the class is loaded (before main or any static use).

Example 1: static variable & method

```
class Counter {
    static int count = 0;
    Counter() { count++; }
    static void showCount() {
        System.out.println("Objects created: " + count);
    }
}

public class Static1 {
    public static void main(String[] args) {
        new Counter(); new Counter(); new Counter();
        Counter.showCount();    // Objects created: 3
    }
}
```

Example 2: static block

```
public class Static2 {
    static int x;
    static {
        System.out.println("Static block executed");
        x = 100;
    }
    public static void main(String[] args) {
        System.out.println("main started");
        System.out.println("x = " + x);
    }
}

// Output order:
// Static block executed
// main started
// x = 100
```

17. final Keyword – Variable, Method, Class

- **final variable** – constant; must be assigned once (blank final can be set in constructor).
- **final method** – cannot be overridden.
- **final class** – cannot be extended (e.g. String, Integer).

Example 1: final variable & method

```
class Base {
    final int MAX = 100;
    final void show() { System.out.println("Base show"); }
}
// class Child extends Base {
//     void show() { } // COMPILER ERROR - cannot override final
// }
public class Final1 {
    public static void main(String[] args) {
        Base b = new Base();
        System.out.println(b.MAX);
        // b.MAX = 200; // COMPILER ERROR
        b.show();
    }
}
```

Example 2: final class & blank final

```
final class Constants {
    final int id;
    Constants(int id) { this.id = id; } // blank final initialized
}
// class Sub extends Constants {} // COMPILER ERROR
public class Final2 {
    public static void main(String[] args) {
        Constants c = new Constants(42);
        System.out.println(c.id);
    }
}
```

18. Encapsulation – Getters & Setters

Encapsulation = binding data and methods together and hiding internal state. Achieved by making fields **private** and providing public **getters** and **setters** (with optional validation). Benefits: control, flexibility, maintainability.

Example 1: Classic getter / setter

```
class BankAccount {
    private double balance;
    public double getBalance() { return balance; }
    public void setBalance(double balance) {
        if (balance >= 0) this.balance = balance;
        else System.out.println("Invalid balance");
    }
    public void deposit(double amt) {
        if (amt > 0) balance += amt;
    }
}

public class Encap1 {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        acc.setBalance(1000);
        acc.deposit(500);
        System.out.println(acc.getBalance()); // 1500
        // acc.balance = -100; // COMPILE ERROR - private
    }
}
```

Example 2: Read-only property (only getter)

```
class Student {
    private final String id;
    private String name;
    Student(String id, String name) {
        this.id = id;
        this.name = name;
    }
    public String getId() { return id; } // no setter
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}

public class Encap2 {
    public static void main(String[] args) {
        Student s = new Student("S101", "Riya");
        System.out.println(s.getId() + " " + s.getName());
        s.setName("Riya Sharma");
        System.out.println(s.getName());
    }
}
```

19. Inheritance – Single Inheritance

Inheritance allows a class (child/subclass) to acquire properties and methods of another class (parent/superclass). Java supports **single inheritance** of classes (one extends clause). Keyword: extends. Benefits: code reuse, polymorphism, “is-a” relationship.

Example 1: Basic single inheritance

```
class Animal {
    void eat() { System.out.println("Animal eats"); }
}
class Dog extends Animal {
    void bark() { System.out.println("Dog barks"); }
}
public class Inherit1 {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat(); // inherited
        d.bark(); // own
    }
}
```

Example 2: Method overriding + super

```
class Vehicle {
    void start() { System.out.println("Vehicle starting"); }
}
class Car extends Vehicle {
    @Override
    void start() {
        super.start(); // call parent version
        System.out.println("Car starting");
    }
}
public class Inherit2 {
    public static void main(String[] args) {
        new Car().start();
    }
}
// Output:
// Vehicle starting
// Car starting
```

20. Inheritance types – Multilevel & Hierarchical

- **Multilevel:** $A \rightarrow B \rightarrow C$ (chain). C inherits from B which inherits from A.
- **Hierarchical:** One parent, multiple children ($A \rightarrow B$ and $A \rightarrow C$).

Java does **not** support multiple inheritance of classes (avoids diamond problem). Interfaces can be used for multiple inheritance of type.

Example 1: Multilevel inheritance

```
class A {
    void showA() { System.out.println("A"); }
}
class B extends A {
    void showB() { System.out.println("B"); }
}
class C extends B {
    void showC() { System.out.println("C"); }
}
public class MultiLevel {
    public static void main(String[] args) {
        C obj = new C();
        obj.showA();
        obj.showB();
        obj.showC();
    }
}
```

Example 2: Hierarchical inheritance

```
class Shape {
    void area() { System.out.println("Area of shape"); }
}
class Circle extends Shape {
    void area() { System.out.println("πr²"); }
}
class Rectangle extends Shape {
    void area() { System.out.println("l × b"); }
}
public class Hierarchical {
    public static void main(String[] args) {
        Shape s1 = new Circle();
        Shape s2 = new Rectangle();
        s1.area();    // πr²
        s2.area();    // l × b
    }
}
```

You now have complete coverage of Java Day 2 topics with explanations and two working programs each. Practice writing the programs yourself and dry-run nested loops / recursion on paper.

All the best for your placements!